```
import React, { useState, useEffect, useCallback } from 'react';
import { initializeApp } from 'firebase/app';
import { getAuth, signInAnonymously, signInWithCustomToken, onAuthStateChanged } from
'firebase/auth';
import { getFirestore, doc, onSnapshot, collection, query, setDoc, updateDoc, arrayUnion,
addDoc } from 'firebase/firestore';
import { ShoppingBag, Ticket, Utensils, Calendar, ChefHat, User, AlertTriangle, Loader,
ChevronLeft, Briefcase, Store, Lock, Globe, Clock, MapPin, ListPlus, Coffee, Milk, Package, Phone,
DollarSign, CheckCircle, Search } from 'lucide-react';


// --- Global Variables (Provided by Canvas Environment) ---
// Initialize these variables defensively as they are provided at runtime.
const appId = typeof __app_id !== 'undefined' ? __app_id : 'uni-e-shop-default-id';
const firebaseConfig = typeof __firebase_config !== 'undefined' ? JSON.parse(__firebase_config) :
null;
const initialAuthToken = typeof __initial_auth_token !== 'undefined' ? __initial_auth_token : null;


// The collection paths for public, shared data in Firestore
const SHOPS_COLLECTION_PATH = `artifacts/${appId}/public/data/shops`;
const TICKETS_COLLECTION_PATH = `artifacts/${appId}/public/data/tickets`;
const SERVICES_COLLECTION_PATH = `artifacts/${appId}/public/data/services`;
const ORDERS_COLLECTION_PATH = `artifacts/${appId}/public/data/orders`; // NEW


// Define all available shop categories (UPDATED: 'Snacks' is renamed and now non-food)
const ShopCategories = [
  'Snacks & Beverages', // UPDATED: Now a non-food category
  'Food Court',         // Remains the only 'food' category
  'Books & Stationery',
  'Electronics & Gadgets',
  'Apparel & Accessories',
  'Services',
  'General Merchandise'
];


// --- Translation Dictionary ---
const Translations = {
  en: {
      // App/General
      title: "Uni E-Shop",
      loading: "Loading Uni E-Shop...",
      back: "Back",
      userId: "User ID:",
      mins: "mins",

      // Roles/Access Screen
      accessTitle: "Access Uni E-Shop",
      roleCustomer: "Student/Customer",
      roleShopkeeper: "Shopkeeper/Vendor",
      iamA: "I am a:",
```

```
    shopkeeperKey: "Shopkeeper Key:",
    enterKeyPlaceholder: "Enter Shopkeeper Key",
    mockKeyInfo: "Mock Key:",
    errorInvalidKey: "Invalid Shopkeeper key.",
    errorCustomerFields: "Please fill in all your details (Name, Phone, Address) to proceed.", // NEW
    yourDetails: "Your Details", // NEW
    namePlaceholder: "Your Full Name", // NEW
    phonePlaceholder: "Phone Number", // NEW
    addressPlaceholder: "Address/Hostel Room No.", // NEW
    buttonCustomerAccess: "Enter Customer Portal",
    buttonShopkeeperAccess: "Log In & Manage Shop",


    // Customer Portal
    selectService: "Select a Campus Service",
    shops: "Shops",
    eventTickets: "Event Tickets",
    food: "Food Court", // UPDATED: Button title is now only "Food Court"
    appointment: "Appointment",
    comingSoon: "Coming Soon!",
    underDevelopment: "The **{featureName}** feature is currently under development. Check back soon for updates!",

    // Food Selection Screen (REMOVED LOGIC)
    selectFoodOption: "Select Ordering Option", // Kept for translation consistency
    optionSnacksBeverages: "Campus Snacks & Beverages", // UPDATED
    optionFoodCourt: "Food Court Stalls",


    // Shops Browser
    exploreShops: "Explore Campus Shops",
    exploreSnacksBeverages: "Explore Campus Snacks & Beverages", // UPDATED
    exploreCourt: "Explore Food Court",
    noShopsTitle: "No shops or items found yet.",
    noShopsHint: "Shopkeepers can log in from the main menu to upload their items!",
    noItems: "This shop has not uploaded any items yet.",
    allCategories: "All Categories",
    searchPlaceholder: "Search items or products...", // NEW

    // Events Browser
    exploreEvents: "Upcoming University Events",
    noEventsTitle: "No tickets found for upcoming events.",
    noEventsHint: "Shopkeepers can upload events from their management portal!",
    buyTicket: "Buy Ticket",


    // Appointment Feature
    appointmentManagement: "Appointment/Service Management",
    manageServices: "Manage Bookable Services",
    serviceUpload: "Upload New Bookable Service",
    serviceName: "Service Name (e.g., Academic Counseling)",
    servicePrice: "Service Price (in INR)",
```

serviceDuration: "Duration (in minutes)",
serviceDescription: "Description",
uploadService: "Upload Service",
exploreAppointments: "Book Campus Services & Appointments",
bookNow: "Book Now",
errorServiceFields: "All service fields are required.",

// Shopkeeper Dashboard - General
shopManagement: "Shop Management",
manageProducts: "Manage Products/Menu",
manageTickets: "Manage Events/Tickets",
manageOrders: "Manage Orders", // NEW
new: "New",
edit: "Edit",
updatePrice: "Update Price",
markAvailable: "Mark Available",
markNotAvailable: "Mark Not Available",
currentItems: "3. Current Menu/Product List",

// Shopkeeper Dashboard - Shop
shopDetails: "1. Shop Details",
selectShopToManage: "Select Shop to Manage:",
createNewShop: "--- Create New Shop ---",
shopNamePlaceholder: "Your Shop Name (e.g., Campus Bites)",
shopCategory: "Shop Category:",
updateShop: "Update Shop Details",
createShop: "Create Shop",
itemUpload: "2. Upload Item (Menu/Product)",
itemNamePlaceholder: "Item Name (e.g., Chicken Roll / Physics Textbook)",
pricePlaceholder: "Price (in INR, e.g., 80.50)",
photoUploadLabel: "Upload Photo (File or Camera)", // NEW
addItem: "Add Item",
errorSaveShop: "Please save your shop name and category first.",
errorItemFields: "All item fields are required (Name, Price, and Photo).", // UPDATED
errorRequiredShop: "Shop name and category are required.",

// Shopkeeper Dashboard - Tickets
ticketManagement: "Ticket Management",
ticketUpload: "Upload New Event Ticket",
eventName: "Event Name (e.g., Annual Cultural Fest)",
eventLocation: "Location (e.g., Auditorium)",
eventDate: "Date",
eventTime: "Time (24h format)",
ticketPrice: "Ticket Price (in INR)",
uploadTicket: "Upload Event Ticket",
errorTicketFields: "All ticket fields are required.",

// Shopkeeper Dashboard - Orders (NEW)
ordersTitle: "Pending Orders for Your Shop(s)",
noNewOrders: "No new orders for your shop.",
customerDetails: "Customer Details",
deliverTo: "Deliver To:",

```
    contact: "Contact:",
    orderId: "Order ID:",
    orderDate: "Order Date:",
    orderStatus: "Status:",

    // Cart/Ordering
    addToCart: "Add to Cart",
    viewCart: "View Cart",
    cartEmpty: "Your cart is empty.",
    cartTotal: "Total:",
    placeOrder: "Place Order", // UPDATED
    startShopping: "Start Shopping",
    paymentMethod: "Payment Method:", // NEW
    cod: "Cash on Delivery (COD)", // NEW
    errorPlaceOrder: "Error placing order. Please try again.",
    errorMissingProfile: "Please complete your profile details (Name, Phone, Address) before placing an order.",

    // Order Confirmation (NEW)
    orderSummaryTitle: "Order Confirmation",
    orderPlacedSuccessfully: "Order Placed Successfully!",
    orderNumber: "Order Number:",
    deliveryInfo: "Delivery Information:",
    estimatedDelivery: "Estimated Delivery:",
    deliveryTime: "30-45 minutes",
    itemBreakdown: "Order Breakdown:",
    backToPortal: "Back to Main Portal",

    // Categories (UPDATED)
    catSnacksBeverages: "Snacks & Beverages", // UPDATED
    catFoodCourt: "Food Court",
    catBooks: "Books & Stationery",
    catElectronics: "Electronics & Gadgets",
    catApparel: "Apparel & Accessories",
    catServices: "Services",
    catGeneral: "General Merchandise"
  },
  hi: {
    // App/General
    title: "[illegible]",
    loading: "[illegible] ...",
    back: "[illegible]",
    userId: "[illegible]",
    mins: "[illegible]",

    // Roles/Access Screen
    accessTitle: "[illegible]",
    roleCustomer: "[illegible]/[illegible]",
    roleShopkeeper: "[illegible]/[illegible]",
    iamA: "[illegible]:",
    shopkeeperKey: "[illegible]:",
    enterKeyPlaceholder: "[illegible]",
    mockKeyInfo: "[illegible]:",
    errorInvalidKey: "[illegible]",
```

```
errorCustomerFields: "[illegible] ([illegible], [illegible]) [illegible]", // NEW
yourDetails: "[illegible]", // NEW
namePlaceholder: "[illegible]", // NEW
phonePlaceholder: "[illegible]", // NEW
addressPlaceholder: "[illegible]/[illegible]", // NEW
buttonCustomerAccess: "[illegible]",
buttonShopkeeperAccess: "[illegible]",

// Customer Portal
selectService: "[illegible]",
shops: "[illegible]",
eventTickets: "[illegible]",
food: "[illegible]", // UPDATED: Button title is now only "[illegible]"
appointment: "[illegible]",
comingSoon: "[illegible]!",
underDevelopment: "**{featureName}** [illegible]
[illegible]!",


// Food Selection Screen (REMOVED LOGIC)
selectFoodOption: "[illegible]",
optionSnacksBeverages: "[illegible]", // UPDATED
optionFoodCourt: "[illegible]",

// Shops Browser
exploreShops: "[illegible]",
exploreSnacksBeverages: "[illegible]", // UPDATED
exploreCourt: "[illegible]",
noShopsTitle: "[illegible]",
noShopsHint: "[illegible]!",
noItems: "[illegible]",
allCategories: "[illegible]",
searchPlaceholder: "[illegible]...", // NEW


// Events Browser
exploreEvents: "[illegible]",
noEventsTitle: "[illegible]",
noEventsHint: "[illegible]!",
buyTicket: "[illegible]",


// Appointment Feature
appointmentManagement: "[illegible]/[illegible]",
manageServices: "[illegible]",
serviceUpload: "[illegible]",
serviceName: "[illegible] ([illegible])",
servicePrice: "[illegible] (INR [illegible])",
serviceDuration: "[illegible] ([illegible])",
serviceDescription: "[illegible]",
uploadService: "[illegible]",
exploreAppointments: "[illegible]",
bookNow: "[illegible]",
```

```
errorServiceFields: "[illegible]",


// Shopkeeper Dashboard - General
shopManagement: "[illegible]",
manageProducts: "[illegible]/[illegible]",
manageTickets: "[illegible]/[illegible]",
manageOrders: "[illegible]", // NEW
new: "[illegible]",
edit: "[illegible]",
updatePrice: "[illegible]",
markAvailable: "[illegible]",
markNotAvailable: "[illegible]",
currentItems: "3. [illegible]/[illegible]",


// Shopkeeper Dashboard - Shop
shopDetails: "1. [illegible]",
selectShopToManage: "[illegible]:",
createNewShop: "--- [illegible] ---",
shopNamePlaceholder: "[illegible] ([illegible]: [illegible])",
shopCategory: "[illegible]:",
updateShop: "[illegible]",
createShop: "[illegible]",
itemUpload: "2. [illegible] ([illegible]/[illegible])",
itemNamePlaceholder: "[illegible] ([illegible]: [illegible]/[illegible])",
pricePlaceholder: "[illegible] (INR [illegible], [illegible] 80.50)",
photoUploadLabel: "[illegible] ([illegible])", // NEW
addItem: "[illegible]",
errorSaveShop: "[illegible]",
errorItemFields: "[illegible] ([illegible], [illegible])", // UPDATED
errorRequiredShop: "[illegible]",


// Shopkeeper Dashboard - Tickets
ticketManagement: "[illegible]",
ticketUpload: "[illegible]",
eventName: "[illegible] ([illegible])",
eventLocation: "[illegible] ([illegible])",
eventDate: "[illegible]",
eventTime: "[illegible] (24 [illegible])",
ticketPrice: "[illegible] (INR [illegible])",
uploadTicket: "[illegible]",
errorTicketFields: "[illegible]",

// Shopkeeper Dashboard - Orders (NEW)
ordersTitle: "[illegible]",
noNewOrders: "[illegible]",
customerDetails: "[illegible]",
deliverTo: "[illegible]:",
contact: "[illegible]:",
orderId: "[illegible]:",
orderDate: "[illegible]:",
```

```
    orderStatus: "□ □ □ □ □ □ :",

    // Cart/Ordering
    addToCart: "□ □ □□□□□□ □ □ □ □ ",
    viewCart: "□ □ □□□□□ □ □ ",
    cartEmpty: "□ □ □□□ □□□□□□□□ □ ",
    cartTotal: "□ □□□ □ □ :",
    placeOrder: "□ □ □□□□□ □ "UPDATED
    startShopping: "□ □ □ □□□□□□□□ □ □ ",
    paymentMethod: "□ □ □ □□□□□ □ :NEW
    cod: "□ □□□□ □ □ □ (COD)", // NEW
    errorPlaceOrder: "□ □ □□□□□□□□□□□ □ □□□□□□□□□□□□□ □ □□□□□□□□ □ ",
    errorMissingProfile: "□ □ □□□□□□□□□ □ □□□ □□□□□□□□□ □ □ □□□□□□□□□(□□□ □□, □□□□, □□) □□□□ □ □ □ ",

    // Order Confirmation (NEW)
    orderSummaryTitle: "□ □ □□□□ □ □ □ □ □ ",
    orderPlacedSuccessfully: "□ □ □□□ □ □ □ □□□□□□□□ □!",
    orderNumber: "□ □ □□□ □ □ □ □ :",
    deliveryInfo: "□ □ □□□□□ □ □ □ □ :",
    estimatedDelivery: "□ □ □ □ □□□□□ □ :",
    deliveryTime: "30-45 □ □ □ □ ",
    itemBreakdown: "□ □ □□□ □ □ □ □ :",
    backToPortal: "□ □ □□□□□ □ □□□□ □□□ □ □ ",

    // Categories (UPDATED)
    catSnacksBeverages: "□ □ □ □□□□□ □ □ "UPDATED
    catFoodCourt: "□ □□□ □ □ □ ",
    catBooks: "□ □ □ □□□□□ □ □ □ □ □ □ ",
    catElectronics: "□ □ □ □ □ □ □ □□□□□□□□□□ □ ",
    catApparel: "□ □ □ □□ □□□□ □□□□□ □ □ ",
    catServices: "□ □ □ □ □ □ ",
    catGeneral: "□ □ □ □□□□□ "
  }
};


// --- Utility Functions ---


// Simple translation getter function
const T_func = (lang, key, replacements = {}) => {
  let text = Translations[lang][key] || key;
  for (const [placeholder, value] of Object.entries(replacements)) {
    text = text.replace(`{${placeholder}}`, value);
  }
  return text;
};


// Utility function to get translated category names
const getCategoryTranslation = (category, lang) => {
  const categoryKeyMap = {
    'Snacks & Beverages': 'catSnacksBeverages', // UPDATED
```

```
    'Food Court': 'catFoodCourt',
    'Books & Stationery': 'catBooks',
    'Electronics & Gadgets': 'catElectronics',
    'Apparel & Accessories': 'catApparel',
    'Services': 'catServices',
    'General Merchandise': 'catGeneral'
  };
  const key = categoryKeyMap[category] || category;
  return Translations[lang][key] || category;
}


// --- View Definitions ---
const Views = {
  DASHBOARD: 'DASHBOARD',
  CUSTOMER_PORTAL: 'CUSTOMER_PORTAL',
  SHOPS: 'SHOPS',
  COURT_LISTING: 'COURT_LISTING',
  TICKETS: 'TICKETS',
  APPOINTMENT: 'APPOINTMENT',
  SHOPKEEPER_DASHBOARD: 'SHOPKEEPER_DASHBOARD',
  CART: 'CART',
  ORDER_CONFIRMATION: 'ORDER_CONFIRMATION', // NEW VIEW
};


// Function to get the customer's private profile document reference
const getUserProfileDocRef = (db, userId) => doc(db, `artifacts/${appId}/users/${userId}/profile/
details`);



// --- Main App Component ---


const App = () => {
  const [db, setDb] = useState(null);
  const [auth, setAuth] = useState(null);
  const [userId, setUserId] = useState(null);
  const [isAuthReady, setIsAuthReady] = useState(false);
  const [currentView, setCurrentView] = useState(Views.DASHBOARD);
  const [shopsData, setShopsData] = useState([]);
  const [ticketsData, setTicketsData] = useState([]);
  const [servicesData, setServicesData] = useState([]);
  const [ordersData, setOrdersData] = useState([]);
  const [cart, setCart] = useState([]);
  const [lastPlacedOrder, setLastPlacedOrder] = useState(null); // NEW STATE
  const [loading, setLoading] = useState(true);
  const [language, setLanguage] = useState('en');

  // UPDATED: State for customer profile (null = not signed up/loaded)
  const [customerProfile, setCustomerProfile] = useState(null);
  const [isProfileLoading, setIsProfileLoading] = useState(false); // New state for profile loading
```

```
// NEW: Ref to track if the authentication check has completed once
const authCheckRef = React.useRef(false);


// Create a translated function bound to the current language
const T = useCallback((key, replacements) => T_func(language, key, replacements), [language]);


// NEW: Function to handle adding items to the cart
const handleAddToCart = useCallback((shop, item, fulfillmentId = null) => {
    setCart(prevCart => {
        // Create a stable unique ID for the cart item using item name and shop ID
        const itemUniqueId = item.name + '_' + shop.id;


        const existingItemIndex = prevCart.findIndex(
            cartItem => cartItem.uniqueId === itemUniqueId
        );


        if (existingItemIndex > -1) {
            // Item found, increase quantity
            const newCart = [...prevCart];
            newCart[existingItemIndex].quantity += 1;
            return newCart;
        } else {
            // Item not found, add new item
            return [...prevCart, {
                uniqueId: itemUniqueId,
                shopId: shop.id,
                shopName: shop.shopName,
                name: item.name,
                price: item.price,
                quantity: 1,
                fulfillmentId: fulfillmentId || shop.id, // Store who fulfills the order
            }];
        }
    });
}, []);


// NEW: Function to handle removing one item from the cart
const handleRemoveOneFromCart = useCallback((itemUniqueId) => {
    setCart(prevCart => {
        const existingItemIndex = prevCart.findIndex(
            cartItem => cartItem.uniqueId === itemUniqueId
        );


        if (existingItemIndex === -1) return prevCart; // Should not happen
```

```
        const newCart = [...prevCart];

        if (newCart[existingItemIndex].quantity > 1) {
            // Decrease quantity
            newCart[existingItemIndex].quantity -= 1;
        } else {
            // Remove item completely if quantity is 1
            newCart.splice(existingItemIndex, 1);
        }
        return newCart;
    });
 }, []);

 // NEW: Function to handle placing the order
 const handlePlaceOrder = useCallback(async () => {
    if (!db || !userId || cart.length === 0) return;


    // 1. Validation check for customer profile
    if (!customerProfile || !customerProfile.name || !customerProfile.phone || !
customerProfile.address) {
        // Using alert here as per previous conversation pattern, though a custom modal is
preferred.
        alert(T('errorMissingProfile'));
        return;
    }


    // 2. Group items by shop for easy fulfillment and shopkeeper filtering
    const itemsByShop = cart.reduce((acc, item) => {
        // Use the stored fulfillmentId (shopId, organizerId, or vendorId) for grouping
        const fulfillmentId = item.fulfillmentId;


        if (!acc[fulfillmentId]) {
            acc[fulfillmentId] = {
                shopName: item.shopName,
                vendorId: fulfillmentId, // Store the vendor/organizer ID here for filtering!
                items: []
            };
        }
        acc[fulfillmentId].items.push({
            name: item.name,
            quantity: item.quantity,
            price: item.price
        });
        return acc;
    }, {});

    // Prepare the final order object
    const newOrder = {
        customerId: userId,
        customerName: customerProfile.name,
```

```
            customerPhone: customerProfile.phone,
            customerAddress: customerProfile.address,
            itemsByShop: Object.values(itemsByShop), // Convert map values to array
            totalAmount: cart.reduce((sum, item) => sum + item.price * item.quantity, 0),
            status: 'Pending',
            timestamp: Date.now(),
            paymentMethod: "Cash on Delivery"
        };

        try {
            // 3. Save order to the public orders collection
            const docRef = await addDoc(collection(db, ORDERS_COLLECTION_PATH), newOrder);

            setCart([]); // Clear cart on success

            // --- NEW: Store order for confirmation view ---
            setLastPlacedOrder({ id: docRef.id, ...newOrder });
            setCurrentView(Views.ORDER_CONFIRMATION);
            // --- END NEW ---

        } catch (error) {
            console.error("Error placing order:", error);
            // Using alert here as per previous conversation pattern.
            alert(T('errorPlaceOrder'));
        }
    }, [db, userId, cart, customerProfile, T]);



    // 1. Firebase Initialization and Authentication
    useEffect(() => {
        if (!firebaseConfig) {
            console.error("Firebase configuration is missing.");
            setLoading(false);
            return;
        }


        const app = initializeApp(firebaseConfig);
        const authInstance = getAuth(app);
        const dbInstance = getFirestore(app);

        // Set DB and Auth instances right away
        setAuth(authInstance);
        setDb(dbInstance);


        let unsubscribeAuth = () => {};


        const initializeAuth = async () => {
            // 1. Attempt initial sign-in FIRST (AWAIT)
            try {
```

```
            if (initialAuthToken) {
                await signInWithCustomToken(authInstance, initialAuthToken);
            } else {
                // Sign in anonymously to ensure an auth context exists for public data reads
                await signInAnonymously(authInstance);
            }
            console.log("Firebase initial sign-in successful.");
        } catch (authError) {
            console.error("Initial sign-in failed:", authError);
        }


        // 2. Set up the ONGOING listener.
        unsubscribeAuth = onAuthStateChanged(authInstance, (newUser) => {
            if (newUser) {
                setUserId(newUser.uid);
            } else if (authInstance.currentUser) {
                setUserId(authInstance.currentUser.uid);
            } else {
                // Fallback UUID for guest if anonymous sign-in failed
                setUserId('guest-' + crypto.randomUUID());
            }


            if (!authCheckRef.current) {
                setIsAuthReady(true);
                setLoading(false);
                authCheckRef.current = true;
            }
        });
    };

    initializeAuth();


    // 3. Return the cleanup function.
    return () => {
        if (typeof unsubscribeAuth === 'function') {
            unsubscribeAuth();
        }
    };
}, []); // Empty dependency array ensures this runs once



// 2. Firestore Listeners for Shops, Tickets, Services, AND Orders (Public Data)
useEffect(() => {
    // IMPORTANT: Wait until Firebase is initialized and auth state is checked
    if (!db || !isAuthReady) return;


    // Shops Listener
    const shopsRef = collection(db, SHOPS_COLLECTION_PATH);
```

```
const unsubscribeShops = onSnapshot(query(shopsRef), (snapshot) => {
    const fetchedShops = [];
    snapshot.forEach((doc) => {
        fetchedShops.push({ id: doc.id, ...doc.data() });
    });
    setShopsData(fetchedShops);
}, (error) => {
    console.error("Error listening to shops data:", error);
});


// Tickets Listener
const ticketsRef = collection(db, TICKETS_COLLECTION_PATH);
const unsubscribeTickets = onSnapshot(query(ticketsRef), (snapshot) => {
    const fetchedTickets = [];
    snapshot.forEach((doc) => {
        fetchedTickets.push({ id: doc.id, ...doc.data() });
    });
    setTicketsData(fetchedTickets);
}, (error) => {
    console.error("Error listening to tickets data:", error);
});

// Services Listener
const servicesRef = collection(db, SERVICES_COLLECTION_PATH);
const unsubscribeServices = onSnapshot(query(servicesRef), (snapshot) => {
    const fetchedServices = [];
    snapshot.forEach((doc) => {
        fetchedServices.push({ id: doc.id, ...doc.data() });
    });
    setServicesData(fetchedServices);
}, (error) => {
    console.error("Error listening to services data:", error);
});

// Orders Listener (NEW)
const ordersRef = collection(db, ORDERS_COLLECTION_PATH);
// Sort by timestamp descending so new orders appear first
const unsubscribeOrders = onSnapshot(query(ordersRef), (snapshot) => {
    const fetchedOrders = [];
    snapshot.forEach((doc) => {
        fetchedOrders.push({ id: doc.id, ...doc.data() });
    });
    // Manual sort by timestamp DESC since orderBy() is avoided
    fetchedOrders.sort((a, b) => b.timestamp - a.timestamp);
    setOrdersData(fetchedOrders);
}, (error) => {
    console.error("Error listening to orders data:", error);
});



return () => {
    unsubscribeShops();
```

```
        unsubscribeTickets();
        unsubscribeServices();
        unsubscribeOrders(); // Clean up orders listener
    };
}, [db, isAuthReady]);

// 3. Customer Profile Listener (Private Data)
useEffect(() => {
    if (!db || !isAuthReady || !userId) return;


    setIsProfileLoading(true);


    const profileRef = getUserProfileDocRef(db, userId);
    const unsubscribeProfile = onSnapshot(profileRef, (docSnapshot) => {
        if (docSnapshot.exists()) {
            // Profile found (logged in / signed up previously)
            setCustomerProfile({ id: docSnapshot.id, ...docSnapshot.data() });
        } else {
            // Profile not found (new user, needs to sign up)
            setCustomerProfile(null);
        }
        setIsProfileLoading(false);
    }, (error) => {
        console.error("Error listening to user profile:", error);
        setIsProfileLoading(false);
        setCustomerProfile(null);
    });


    return () => unsubscribeProfile();


}, [db, isAuthReady, userId]);



// --- Shared Components ---


const LoadingState = () => (
    <div className="flex flex-col items-center justify-center p-8 text-indigo-500">
        <Loader className="w-8 h-8 animate-spin" />
        <p className="mt-4 text-lg font-semibold">{T('loading')}</p>
    </div>
);


const getBackView = (current) => {
    switch (current) {
        case Views.SHOPS:
        case Views.TICKETS:
```

```
        case Views.APPOINTMENT:
        case Views.CART:
        case Views.COURT_LISTING:
        case Views.ORDER_CONFIRMATION: // Back from confirmation goes to portal
          return Views.CUSTOMER_PORTAL;
        case Views.CUSTOMER_PORTAL:
        case Views.SHOPKEEPER_DASHBOARD:
          return Views.DASHBOARD;
        default:
          return Views.DASHBOARD;
      }
    }


  const Header = ({ title }) => (
      <header className="p-4 bg-indigo-700 shadow-xl rounded-b-lg">
        <div className="flex items-center justify-between">
          {currentView !== Views.DASHBOARD && (
            <button
              onClick={() => setCurrentView(getBackView(currentView))}
              className="flex items-center text-white p-2 rounded-full hover:bg-indigo-600
transition duration-150"
            >
              <ChevronLeft className="w-5 h-5 mr-1" />
              {T('back')}
            </button>
          )}
          <h1 className={`text-2xl font-extrabold text-white tracking-tight ${currentView ===
Views.DASHBOARD ? 'w-full text-center' : ''}`}>
            {title}
          </h1>
          <div className="flex items-center space-x-4">
            {/* Cart Icon (only visible for customer-facing views) */}
            {currentView !== Views.DASHBOARD && currentView !==
Views.SHOPKEEPER_DASHBOARD && (
              <button
                onClick={() => setCurrentView(Views.CART)}
                className="relative text-white p-2 rounded-full hover:bg-indigo-600 transition
duration-150"
              >
                <ShoppingBag className="w-5 h-5" />
                {cart.length > 0 && (
                  <span className="absolute -top-1 -right-1 bg-red-500 text-white text-xs font-
bold rounded-full w-5 h-5 flex items-center justify-center">
                    {cart.reduce((sum, item) => sum + item.quantity, 0)}
                  </span>
                )}
              </button>
            )}


            {/* Language Selector */}
            <div className="flex items-center text-white">
              <Globe className="w-4 h-4 mr-1" />
```

```
                <select
                  value={language}
                  onChange={(e) => setLanguage(e.target.value)}
                  className="bg-indigo-700 text-white text-sm font-medium border-none
focus:ring-0 focus:outline-none cursor-pointer"
                >
                  <option value="en" className="text-gray-900">English</option>
                  <option value="hi" className="text-gray-900">□ □ □ □(Hindi)</option>
                </select>
              </div>


              <div className="text-sm font-medium text-white/80 hidden sm:block">
                <span className="hidden lg:inline">{T('userId')}</span> {userId ?
userId.substring(0, 8) + '...' : 'Guest'}
              </div>
            </div>
          </div>
        </header>
      );


      const CategoryCard = ({ icon: Icon, title, onClick, color = 'bg-indigo-500' }) => (
        <button
          onClick={onClick}
          className={`flex flex-col items-center justify-center p-6 ${color} text-white rounded-xl
shadow-lg hover:shadow-2xl transition duration-300 transform hover:scale-[1.02] active:scale-
[0.98] w-full h-36`}
        >
          <Icon className="w-8 h-8 mb-2" />
          <span className="text-lg font-bold">{title}</span>
        </button>
      );


      const NotYetImplemented = ({ featureName }) => (
        <div className="p-6 text-center text-gray-700 bg-yellow-100 rounded-xl shadow-inner">
          <AlertTriangle className="w-6 h-6 mx-auto mb-3 text-yellow-600" />
          <h2 className="text-xl font-semibold">{T('comingSoon')}</h2>
          <p className="mt-2" dangerouslySetInnerHTML={{ __html: T('underDevelopment',
{ featureName }) }}></p>
        </div>
      );


      // NEW: Component to handle customer sign-up/login persistence
      const CustomerAccess = ({ isProfileLoading, customerProfile, db, userId, T, setCurrentView }) =>
{
        const [customerName, setCustomerName] = useState(customerProfile ?
customerProfile.name : '');
        const [customerPhone, setCustomerPhone] = useState(customerProfile ?
customerProfile.phone : '');
        const [customerAddress, setCustomerAddress] = useState(customerProfile ?
```

```
customerProfile.address : '');
    const [isSaving, setIsSaving] = useState(false);
    const [saveError, setSaveError] = useState('');

    // Update local state if customerProfile changes (e.g., loaded by listener)
    useEffect(() => {
        if (customerProfile) {
            setCustomerName(customerProfile.name || '');
            setCustomerPhone(customerProfile.phone || '');
            setCustomerAddress(customerProfile.address || '');
        }
    }, [customerProfile]);



    const handleSaveProfile = async (e) => {
        e.preventDefault();
        setSaveError('');

        if (!customerName || !customerPhone || !customerAddress) {
            setSaveError(T('errorCustomerFields'));
            return;
        }


        if (!db || !userId) {
            setSaveError('App initialization failed. Please reload.');
            return;
        }


        setIsSaving(true);
        try {
            const profileRef = getUserProfileDocRef(db, userId);
            const profileData = {
                name: customerName,
                phone: customerPhone,
                address: customerAddress,
                lastUpdated: Date.now()
            };


            // Use setDoc to create or overwrite the profile
            await setDoc(profileRef, profileData);

            // Once saved, navigate to the portal
            setCurrentView(Views.CUSTOMER_PORTAL);

        } catch (error) {
            console.error("Error saving profile:", error);
            setSaveError(`Failed to save profile: ${error.message}`);
        } finally {
            setIsSaving(false);
```

```
    }
  };



  if (isProfileLoading) {
    return (
      <div className="flex flex-col items-center justify-center py-6 text-indigo-500">
        <Loader className="w-6 h-6 animate-spin" />
        <p className="mt-2 text-sm">{T('loading')}</p>
      </div>
    );
  }



  if (customerProfile) {
    // Logged In / Profile Found (Welcome Back Screen)
    return (
      <div className="py-4 space-y-4 text-center">
        <h3 className="text-xl font-bold text-green-600">Welcome Back,
{customerProfile.name}!</h3>
        <p className="text-sm text-gray-600">Your details are saved.
({customerProfile.phone})</p>
        <button
          onClick={() => setCurrentView(Views.CUSTOMER_PORTAL)}
          className="w-full p-3 rounded-lg font-semibold transition duration-200 bg-
indigo-500 hover:bg-indigo-600 text-white"
        >
          {T('buttonCustomerAccess')}
        </button>
        <button
          onClick={() => setCustomerProfile(null)} // Allows user to re-sign up if data is wrong
          className="text-sm text-indigo-500 hover:text-indigo-700 underline"
        >
          Change/Update Details
        </button>
      </div>
    );
  }



  // Sign Up / Profile Not Found (Sign Up Form)
  // This form is intended to be rendered inside the AccessScreen's main div, NOT inside
another form.
  return (
    <form onSubmit={handleSaveProfile} className="space-y-4">
      <div className="space-y-3 p-3 border border-green-300 rounded-xl bg-green-50">
        <h3 className="text-base font-semibold text-green-700">{T('yourDetails')} (Sign Up)</
h3>
        <input
          type="text"
          placeholder={T('namePlaceholder')}
          value={customerName}
```

```
                onChange={(e) => setCustomerName(e.target.value)}
                className="w-full p-3 border border-gray-300 rounded-lg focus:ring-green-500
focus:border-green-500"
                required
              />
              <input
                type="tel"
                placeholder={T('phonePlaceholder')}
                value={customerPhone}
                onChange={(e) => setCustomerPhone(e.target.value)}
                className="w-full p-3 border border-gray-300 rounded-lg focus:ring-green-500
focus:border-green-500"
                required
              />
              <input
                type="text"
                placeholder={T('addressPlaceholder')}
                value={customerAddress}
                onChange={(e) => setCustomerAddress(e.target.value)}
                className="w-full p-3 border border-gray-300 rounded-lg focus:ring-green-500
focus:border-green-500"
                required
              />
            </div>

            {saveError && <p className="text-red-500 text-sm font-medium">{saveError}</p>}

            <button
              type="submit"
              disabled={isSaving}
              className="w-full p-3 rounded-lg font-semibold transition duration-200 bg-green-500
hover:bg-green-600 text-white disabled:bg-green-300"
            >
              {isSaving ? 'Saving Profile...' : T('buttonCustomerAccess')}
            </button>
          </form>
        );
    };



    // --- Role Access Screen (New Dashboard) ---
    const AccessScreen = () => {
        const [selectedRole, setSelectedRole] = useState('Customer');
        const [password, setPassword] = useState('');
        const [error, setError] = useState('');
        const MOCK_SHOPKEEPER_KEY = 'uni-vendor-2024';

        const handleAccess = (e) => {
            e.preventDefault();
            setError('');


            if (selectedRole === 'Shopkeeper') {
```

```
            if (password === MOCK_SHOPKEEPER_KEY) {
                setCurrentView(Views.SHOPKEEPER_DASHBOARD);
                setPassword('');
            } else {
                setError(T('errorInvalidKey'));
                setPassword('');
            }
        }
    };


    return (
        <div className="max-w-md mx-auto p-6 bg-white rounded-xl shadow-2xl mt-10">
            <h2 className="text-3xl font-bold text-indigo-700 mb-6 text-center flex items-center
justify-center">
                <Lock className="w-6 h-6 mr-2" /> {T('accessTitle')}
            </h2>
            <div className="space-y-5">
                {/* Role Selection (Kept outside any conditional form) */}
                <div>
                    <label className="block text-sm font-medium text-gray-700 mb-1">{T('iamA')}</
label>
                    <select
                        value={selectedRole}
                        onChange={(e) => {
                            setSelectedRole(e.target.value);
                            setError(''); // Clear error on role change
                        }}
                        className="w-full p-3 border border-gray-300 rounded-lg focus:ring-indigo-500
focus:border-indigo-500 transition duration-150 bg-white"
                    >
                        <option value="Customer">{T('roleCustomer')}</option>
                        <option value="Shopkeeper">{T('roleShopkeeper')}</option>
                    </select>
                </div>


                {/* Customer Access (Renders its OWN form internally - no change needed here) */}
                {selectedRole === 'Customer' ? (
                    <CustomerAccess
                        isProfileLoading={isProfileLoading}
                        customerProfile={customerProfile}
                        db={db}
                        userId={userId}
                        T={T}
                        setCurrentView={setCurrentView}
                    />
                ) : (
                    // Shopkeeper Access (Now wrapped in a NEW form)
                    <form onSubmit={handleAccess} className="space-y-5">
                        {/* Password/Key Input for Shopkeeper */}
                        <div>
                            <label className="block text-sm font-medium text-gray-700
mb-1">{T('shopkeeperKey')}</label>
```

```
                        <input
                            type="password"
                            placeholder={T('enterKeyPlaceholder')}
                            value={password}
                            onChange={(e) => setPassword(e.target.value)}
                            className="w-full p-3 border border-gray-300 rounded-lg focus:ring-
indigo-500 focus:border-indigo-500"
                            required
                        />
                        <p className="text-xs text-gray-500 mt-1">{T('mockKeyInfo')}
**`{MOCK_SHOPKEEPER_KEY}`**</p>
                    </div>

                    {error && <p className="text-red-500 text-sm font-medium">{error}</p>}


                    <button
                        type="submit"
                        className="w-full p-3 rounded-lg font-semibold transition duration-200 bg-
green-500 hover:bg-green-600 text-white"
                    >
                        {T('buttonShopkeeperAccess')}
                    </button>
                </form>
            )}
        </div>
    </div>
  );
 };

 // --- Shopkeeper Dashboard Logic ---

 // NEW: Component for managing orders
 const OrderManagement = () => {
    // Filter orders that contain at least one item GROUP where the vendorId matches the current
user ID
    const vendorOrders = ordersData.filter(order => {
        return order.itemsByShop.some(shopItem => {
            // Check if the item group's vendorId matches the current logged-in shopkeeper's userId
            return shopItem.vendorId === userId;
        });
    });


    // Helper to format timestamp
    const formatTimestamp = (timestamp) => {
        return new Date(timestamp).toLocaleString(language, { dateStyle: 'short', timeStyle:
'short' });
    };


    return (
        <div className="p-4 bg-white rounded-xl shadow-lg border-t-4 border-indigo-500">
```

```
        <h3 className="text-xl font-bold text-indigo-700 mb-4">{T('ordersTitle')}</h3>

        {vendorOrders.length === 0 ? (
          <div className="text-center py-6 text-gray-500 italic">
            <Package className="w-8 h-8 mx-auto mb-2" />
            <p>{T('noNewOrders')}</p>
          </div>
        ) : (
          <div className="space-y-6">
            {vendorOrders.map(order => (
              <div key={order.id} className="p-4 border border-gray-200 rounded-lg shadow-sm
bg-gray-50">

                {/* Order Header */}
                <div className="flex justify-between items-start border-b pb-3 mb-3">
                  <div>
                    <p className="text-xs text-gray-500">{T('orderId')} {order.id.substring(0, 8)}
</p>
                    <p className="text-xs text-gray-500">{T('orderDate')}
{formatTimestamp(order.timestamp)}</p>
                  </div>
                  <span className="text-lg font-bold text-red-600">{T('orderStatus')}
{order.status}</span>
                </div>

                {/* Customer Details (Delivery Info) - CRUCIAL for fulfillment */}
                <h4 className="font-semibold text-lg mb-2 text-indigo-700 flex items-center">
                  <User className="w-4 h-4 mr-1" /> {T('customerDetails')}
                </h4>
                <div className="space-y-1 text-sm mb-4 p-3 bg-indigo-50 rounded-lg">
                  <div className="flex items-center">
                    <MapPin className="w-4 h-4 mr-2 text-indigo-500" />
                    <span className="font-medium text-gray-800">{T('deliverTo')}
**{order.customerName}** ({order.customerAddress})</span>
                  </div>
                  <div className="flex items-center">
                    <Phone className="w-4 h-4 mr-2 text-indigo-500" />
                    <span className="font-medium text-gray-800">{T('contact')}
{order.customerPhone}</span>
                  </div>
                </div>

                {/* Payment Method */}
                <div className="flex items-center justify-between text-sm mb-4 border-t pt-2">
                  <span className="font-semibold text-gray-700 flex items-
center"><DollarSign className="w-4 h-4 mr-1 text-green-600"/> {T('paymentMethod')}</span>
                  <span className="font-bold text-green-700">{order.paymentMethod}</span>
                </div>



                {/* Items for This Vendor */}
                <h4 className="font-semibold text-base mb-2 text-gray-700">Fulfillment
Items:</h4>
                <div className="space-y-2">
```

```
                        {/* Filter groups by the current vendor's ID */}
                        {order.itemsByShop
                           .filter(shopItem => shopItem.vendorId === userId)
                           .map(shopItem => (
                              <div key={shopItem.vendorId} className="border-l-2 border-green-500
pl-3">
                                 <p className="font-bold text-sm text-gray-800
mb-1">{shopItem.shopName}</p>
                                 <ul className="list-disc ml-4 text-sm text-gray-600 space-y-0.5">
                                    {shopItem.items.map((item, index) => (
                                       <li key={index}>
                                          {item.quantity} x {item.name} (₹{item.price.toFixed(2)})
                                       </li>
                                    ))}
                                 </ul>
                              </div>
                           ))}
                     </div>

                     <p className="mt-4 pt-3 border-t font-extrabold text-right text-lg">
                        Total Order Value: ₹{order.totalAmount.toFixed(2)}
                     </p>
                  </div>
               ))}
            </div>
         )}
      </div>
   );
};


const ShopProductManagementForm = () => {
   // Find all shops owned by the current user
   const ownedShops = shopsData.filter(s => s.ownerId === userId);

   // State to track which shop the vendor is currently managing
   // 'new' signifies the 'Create New Shop' option is selected
   const [selectedShopId, setSelectedShopId] = useState('new');

   // Find the current shop object based on the selected ID
   const currentShop = ownedShops.find(s => s.id === selectedShopId);


   // Local state for shop details (used for display and creation)
   const [shopName, setShopName] = useState('');
   const [category, setCategory] = useState('Snacks & Beverages');
   const [itemName, setItemName] = useState('');
   const [itemPrice, setItemPrice] = useState('');
   const [message, setMessage] = useState('');

   const [uploadedImageBase64, setUploadedImageBase64] = useState('');

   const [editingItem, setEditingItem] = useState(null);
   const [newPrice, setNewPrice] = useState('');
```

```
    // --- EFFECT: Sync local state when selectedShopId changes ---
    useEffect(() => {
        if (currentShop) {
            // Existing Shop Selected: Load its data
            setShopName(currentShop.shopName);
            const existingCategory = currentShop.category;
            const initialCategory = ShopCategories.includes(existingCategory) ? existingCategory :
'Snacks & Beverages';
            setCategory(initialCategory);
            setMessage(`You are managing: <b>${currentShop.shopName}</b>. Category: <b>
${getCategoryTranslation(initialCategory, language)}</b>.`);
        } else {
            // 'Create New Shop' Selected: Reset local states
            setShopName('');
            setCategory('Snacks & Beverages');
            setMessage(`${T('new')}: ${T('createShop')}. Select details below to begin.`);
        }
        // Clear item-specific states when switching shops
        setItemName('');
        setItemPrice('');
        setUploadedImageBase64('');
        setEditingItem(null);


    }, [selectedShopId, shopsData, language, T, currentShop]);

    // NEW: Handle image file selection and convert to Base64
    const handleImageUpload = (e) => {
        const file = e.target.files[0];
        if (file) {
            const reader = new FileReader();
            reader.onloadend = () => {
                setUploadedImageBase64(reader.result);
                setMessage(`Image loaded: ${file.name}`);
            };
            reader.onerror = () => {
                setMessage('Error reading file.');
                setUploadedImageBase64('');
            };
            reader.readAsDataURL(file);
        }
    };


    // Function to find and update a specific item's properties in Firestore
    const handleUpdateItem = useCallback(async (itemNameKey, updates) => {
        if (!selectedShopId || selectedShopId === 'new' || !db || !currentShop) return;


        try {
            // Ensure the item array exists and find the item by name
            const items = currentShop.items || [];
```

```
        const updatedItems = items.map(item =>
            item.name === itemNameKey ? { ...item, ...updates } : item
        );


        const shopRef = doc(db, SHOPS_COLLECTION_PATH, selectedShopId);
        await updateDoc(shopRef, { items: updatedItems });


        setMessage(`Item "${itemNameKey}" updated successfully.`);
    } catch (error) {
        console.error("Error updating item:", error);
        setMessage(`Error updating item: ${error.message}`);
    }
}, [selectedShopId, db, currentShop]);

// Function to toggle the available status
const handleToggleAvailability = (item) => {
    handleUpdateItem(item.name, { available: !item.available });
};

// Handle price modal initialization
useEffect(() => {
    if (editingItem) {
        // Initialize price input with current price
        setNewPrice(editingItem.price.toFixed(2));
    }
}, [editingItem]);


const handlePriceUpdate = async () => {
    if (!editingItem || isNaN(parseFloat(newPrice)) || parseFloat(newPrice) <= 0) return;


    await handleUpdateItem(editingItem.name, { price: parseFloat(newPrice) });
    setEditingItem(null); // Close modal
    setNewPrice('');
};



// Save/Update Shop Name and Category
const handleSaveShop = async () => {
    if (!shopName || !db || !userId || !category) {
        setMessage(T('errorRequiredShop'));
        return;
    }


    try {
        if (selectedShopId !== 'new') {
            // Update existing shop
            const shopRef = doc(db, SHOPS_COLLECTION_PATH, selectedShopId);
```

```
            await updateDoc(shopRef, { shopName, category });
            setMessage(`Success! ${shopName} updated as <b>
${getCategoryTranslation(category, language)}</b>.`);
        } else {
            // Create new shop
            const newDocRef = doc(collection(db, SHOPS_COLLECTION_PATH));
            await setDoc(newDocRef, {
                shopName,
                ownerId: userId,
                category: category,
                items: []
            });
            // IMPORTANT: After creation, select the new shop ID
            setSelectedShopId(newDocRef.id);
            setMessage(`Success! "${shopName}" created as <b>
${getCategoryTranslation(category, language)}</b>!`);
        }
    } catch (error) {
        console.error("Error saving shop:", error);
        setMessage(`Error saving shop: ${error.message}`);
    }
};


// Add a Food Item/General Product to the shop
const handleAddItem = async (e) => {
    e.preventDefault();

    if (selectedShopId === 'new' || !currentShop) {
        setMessage(T('errorSaveShop'));
        return;
    }
    if (!itemName || !itemPrice || !uploadedImageBase64) {
        setMessage(T('errorItemFields'));
        return;
    }

    // FIX: Strip the MIME prefix from the Base64 data to minimize data size
    const base64Data = uploadedImageBase64.split(',')[1] || uploadedImageBase64;


    // SAFETY CHECK: Limit Base64 string size to ~500KB (3/4 of Firebase 1MB limit for safety
margin)
    const MAX_BASE64_LENGTH = 700000; // Represents roughly 500KB of raw image data


    if (base64Data.length > MAX_BASE64_LENGTH) {
        setMessage('Error: The uploaded image is too large. Please use a smaller file (max
~500KB).');
        return;
    }
```

```
        const newItem = {
            name: itemName,
            price: parseFloat(itemPrice),
            photoUrl: base64Data,
            timestamp: Date.now(),
            available: true
        };


        try {
            const shopRef = doc(db, SHOPS_COLLECTION_PATH, selectedShopId);

            // Read the existing items array
            const currentItems = currentShop.items || []

            // Check for duplicate name
            if (currentItems.some(item => item.name === newItem.name)) {
                setMessage(`Item "${newItem.name}" already exists. Please update the existing
item.`);
                return;
            }


            const updatedItems = [...currentItems, newItem];


            await updateDoc(shopRef, {
                items: updatedItems
            });

            setMessage(`${T('addItem')}: ${itemName} (₹${itemPrice})`);
            setItemName('');
            setItemPrice('');
            setUploadedImageBase64(''); // Clear Base64 state
        } catch (error) {
            console.error("Error adding item:", error);
            setMessage(`Error adding item: ${error.message}`);
        }
    };

    // Component to list current items and provide edit controls
    const CurrentItemsList = () => {
        const items = currentShop?.items || [];

        return (
            <div className="bg-white p-6 rounded-xl shadow-lg border-t-4 border-yellow-500 mt-6">
                <h3 className="text-xl font-semibold mb-3 text-gray-700">{T('currentItems')}</h3>
                <div className="space-y-3">
                    {items.length === 0 ? (
                        <p className="text-gray-500 italic">No items uploaded yet.</p>
                    ) : (
                        items.map((item) => (
                            <div key={item.name} className={`bg-gray-50 p-3 rounded-lg flex flex-col
```

```
sm:flex-row items-start sm:items-center justify-between shadow-sm transition duration-150
${item.available ? 'border-l-4 border-green-500' : 'border-l-4 border-red-500 bg-red-50'}`}>
                        <div className="flex items-center space-x-3 flex-grow mb-2 sm:mb-0">
                          <img
                            // FIX: Re-add prefix for display
                            src={`${item.photoUrl}`}
                            alt={item.name}
                            className="w-10 h-10 object-cover rounded-md"
                            onError={(e) => { e.target.onerror = null; e.target.src = `https://
placehold.co/40x40/3B82F6/white?text=${item.name.substring(0, 1)}` }}
                          />
                          <div>
                            <p className={`font-semibold ${item.available ? 'text-gray-800' : 'text-
red-700 line-through'}`}>{item.name}</p>
                            <p className="text-sm font-bold text-green-600">
₹{item.price.toFixed(2)}</p>
                            <span className={`text-xs px-2 py-0.5 rounded-full ${item.available ?
'bg-green-200 text-green-800' : 'bg-red-200 text-red-800'}`}>
                              {item.available ? 'Available' : 'Not Available'}
                            </span>
                          </div>
                        </div>

                        <div className="flex space-x-2 flex-shrink-0">
                          {/* Button to update price */}
                          <button
                            onClick={() => setEditingItem(item)}
                            className="bg-indigo-500 text-white text-sm px-3 py-1 rounded-full
hover:bg-indigo-600 transition duration-150"
                          >
                            {T('updatePrice')}
                          </button>
                          {/* Button to toggle availability */}
                          <button
                            onClick={() => handleToggleAvailability(item)}
                            className={`text-white text-sm px-3 py-1 rounded-full transition
duration-150 ${item.available ? 'bg-red-500 hover:bg-red-600' : 'bg-green-500 hover:bg-green-600'}
`}
                          >
                            {item.available ? T('markNotAvailable') : T('markAvailable')}
                          </button>
                        </div>
                      </div>
                    ))
                  )}
                </div>

                {/* Price Edit Modal/Inline Form */}
                {editingItem && (
                  <div className="fixed inset-0 bg-black bg-opacity-50 flex items-center justify-center
z-50 p-4">
                    <div className="bg-white p-6 rounded-xl shadow-2xl max-w-sm w-full">
                      <h4 className="text-xl font-bold text-indigo-700 mb-4">{T('updatePrice')}:
{editingItem.name}</h4>
```

```
                    <p className="mb-3 text-sm text-gray-600">Current Price:
₹{editingItem.price.toFixed(2)}</p>
                    <input
                      type="number"
                      step="0.01"
                      placeholder={T('pricePlaceholder')}
                      value={newPrice}
                      onChange={(e) => setNewPrice(e.target.value)}
                      className="w-full p-3 border border-gray-300 rounded-lg mb-4 focus:ring-
indigo-500 focus:border-indigo-500"
                    />
                    <div className="flex justify-end space-x-3">
                      <button
                        onClick={() => setEditingItem(null)}
                        className="bg-gray-300 text-gray-800 px-4 py-2 rounded-lg hover:bg-
gray-400"
                      >
                        Cancel
                      </button>
                      <button
                        onClick={handlePriceUpdate}
                        className="bg-indigo-500 text-white px-4 py-2 rounded-lg hover:bg-
indigo-600 disabled:bg-indigo-300"
                        disabled={isNaN(parseFloat(newPrice)) || parseFloat(newPrice) <= 0}
                      >
                        {T('updatePrice')}
                      </button>
                    </div>
                  </div>
                </div>
              )}
            </div>
        );
    };



    return (
      <>
        <p className="text-green-600 font-medium mb-4" dangerouslySetInnerHTML={{ __html:
message }}></p>

        {/* --- NEW: SHOP SELECTOR --- */}
        <div className="bg-white p-6 rounded-xl shadow-lg mb-6 border-t-4 border-blue-500">
          <label className="block text-sm font-medium text-gray-700 mb-1">
            {T('selectShopToManage')}
          </label>
          <select
            value={selectedShopId}
            onChange={(e) => setSelectedShopId(e.target.value)}
            className="w-full p-3 border border-gray-300 rounded-lg focus:ring-blue-500
focus:border-blue-500 transition duration-150 bg-white font-semibold"
          >
            <option value="new" className="font-bold">{T('createNewShop')}</option>
```

```
            {ownedShops.map(shop => (
              <option key={shop.id} value={shop.id}>
                {shop.shopName} ({getCategoryTranslation(shop.category, language)})
              </option>
            ))}
          </select>
        </div>
        {/* --- END NEW --- */}


        {/* Shop Name and Category Management */}
        <div className="bg-white p-6 rounded-xl shadow-lg mb-6 border-t-4 border-indigo-500">
          <h3 className="text-xl font-semibold mb-3 text-gray-700">{T('shopDetails')}</h3>
          <div className="flex flex-col sm:flex-row gap-4 mb-4">
            <input
              type="text"
              placeholder={T('shopNamePlaceholder')}
              value={shopName}
              onChange={(e) => setShopName(e.target.value)}
              className="flex-grow p-3 border border-gray-300 rounded-lg focus:ring-
indigo-500 focus:border-indigo-500"
            />
            <button
              onClick={handleSaveShop}
              className="bg-indigo-500 text-white p-3 rounded-lg font-semibold hover:bg-
indigo-600 transition duration-200"
            >
              {selectedShopId !== 'new' ? T('updateShop') : T('createShop')}
            </button>
          </div>
          {/* Category Selection Dropdown */}
          <div className="flex flex-col gap-2">
            <label className="text-sm font-medium text-gray-600">{T('shopCategory')}</label>
            <select
              value={category}
              onChange={(e) => setCategory(e.target.value)}
              className="p-3 border border-gray-300 rounded-lg focus:ring-indigo-500
focus:border-indigo-500"
            >
              {ShopCategories.map(cat => (
                <option key={cat} value={cat}>{getCategoryTranslation(cat, language)}</option>
              ))}
            </select>
          </div>
        </div>


        {/* General Item Upload (Conditional on shop selection) */}
        {selectedShopId !== 'new' && (
          <div className="bg-white p-6 rounded-xl shadow-lg border-t-4 border-green-500">
            <h3 className="text-xl font-semibold mb-3 text-gray-700">{T('itemUpload')}</h3>
            <form onSubmit={handleAddItem} className="space-y-4">
              <input
                type="text"
```

```
                placeholder={T('itemNamePlaceholder')}
                value={itemName}
                onChange={(e) => setItemName(e.target.value)}
                className="w-full p-3 border border-gray-300 rounded-lg"
                required
              />
              <div className="flex flex-col sm:flex-row gap-4">
                <input
                  type="number"
                  placeholder={T('pricePlaceholder')}
                  value={itemPrice}
                  onChange={(e) => setItemPrice(e.target.value)}
                  className="w-full sm:w-1/2 p-3 border border-gray-300 rounded-lg"
                  step="0.01"
                  required
                />
                {/* NEW: File/Camera Input */}
                <div className="w-full sm:w-1/2">
                  <label className="block text-sm font-medium text-gray-700
mb-1">{T('photoUploadLabel')}</label>
                  <input
                    type="file"
                    accept="image/*"
                    capture="camera"
                    onChange={handleImageUpload}
                    className="w-full p-2 border border-gray-300 rounded-lg text-sm file:mr-4
file:py-1 file:px-2 file:rounded-md file:border-0 file:text-sm file:font-semibold file:bg-indigo-50
file:text-indigo-700 hover:file:bg-indigo-100"
                    />
                </div>
              </div>

              {/* Image Preview */}
              {uploadedImageBase64 && (
                <div className="mt-2 p-3 bg-gray-100 rounded-lg flex items-center space-x-3">
                  <img src={uploadedImageBase64} alt="Preview" className="w-12 h-12
object-cover rounded-md border border-gray-300"/>
                  <span className="text-sm text-green-600">Image loaded for upload.</span>
                </div>
              )}


              <button
                type="submit"
                className="w-full bg-green-500 text-white p-3 rounded-lg font-semibold
hover:bg-green-600 transition duration-200"
              >
                {T('addItem')}
              </button>
            </form>
          </div>
        )}

        {/* Item Management List */}
```

```
        {selectedShopId !== 'new' && <CurrentItemsList />}
      </>
    );
}

// Component for managing ticket uploads
const TicketManagementForm = () => {
    const [eventName, setEventName] = useState('');
    const [eventLocation, setEventLocation] = useState('');
    const [eventDate, setEventDate] = useState('');
    const [eventTime, setEventTime] = useState('');
    const [ticketPrice, setTicketPrice] = useState('');
    const [photoUrl, setPhotoUrl] = useState('');
    const [message, setMessage] = useState('');


    const handleTicketUpload = async (e) => {
        e.preventDefault();
        if (!eventName || !eventLocation || !eventDate || !eventTime || !ticketPrice || !photoUrl) {
            setMessage(T('errorTicketFields'));
            return;
        }


        const newTicket = {
            eventName,
            eventLocation,
            eventDate,
            eventTime,
            ticketPrice: parseFloat(ticketPrice),
            photoUrl,
            organizerId: userId, // CRUCIAL: Store the user ID of the organizer
            timestamp: Date.now()
        };


        try {
            // Add the new ticket document to the tickets collection
            await addDoc(collection(db, TICKETS_COLLECTION_PATH), newTicket);

            setMessage(`Successfully uploaded ticket for: ${eventName}`);
            setEventName('');
            setEventLocation('');
            setEventDate('');
            setEventTime('');
            setTicketPrice('');
            setPhotoUrl('');


        } catch (error) {
            console.error("Error adding ticket:", error);
            setMessage(`Error adding ticket: ${error.message}`);
        }
```

```
    };


    return (
      <div className="bg-white p-6 rounded-xl shadow-lg border-t-4 border-red-500">
        <h3 className="text-xl font-semibold mb-4 text-gray-700">{T('ticketUpload')}</h3>

        <p className="text-green-600 font-medium mb-4">{message}</p>


        <form onSubmit={handleTicketUpload} className="space-y-4">
          <input type="text" placeholder={T('eventName')} value={eventName} onChange={(e) =>
setEventName(e.target.value)} className="w-full p-3 border border-gray-300 rounded-lg"
required />
          <input type="text" placeholder={T('eventLocation')} value={eventLocation}
onChange={(e) => setEventLocation(e.target.value)} className="w-full p-3 border border-
gray-300 rounded-lg" required />

          <div className="flex gap-4">
            <input type="date" placeholder={T('eventDate')} value={eventDate} onChange={(e) =>
setEventDate(e.target.value)} className="w-1/2 p-3 border border-gray-300 rounded-lg"
required />
            <input type="time" placeholder={T('eventTime')} value={eventTime} onChange={(e)
=> setEventTime(e.target.value)} className="w-1/2 p-3 border border-gray-300 rounded-lg"
required />
          </div>

          <input type="number" placeholder={T('ticketPrice')} value={ticketPrice} onChange={(e)
=> setTicketPrice(e.target.value)} className="w-full p-3 border border-gray-300 rounded-lg"
step="0.01" required />
          <input type="url" placeholder={T('photoUrlPlaceholder')} value={photoUrl}
onChange={(e) => setPhotoUrl(e.target.value)} className="w-full p-3 border border-gray-300
rounded-lg" required />


          <button
            type="submit"
            className="w-full bg-red-500 text-white p-3 rounded-lg font-semibold hover:bg-
red-600 transition duration-200"
          >
            {T('uploadTicket')}
          </button>
        </form>
      </div>
    );
  }

  // NEW: Component for managing bookable services
  const AppointmentManagementForm = () => {
    const [serviceName, setServiceName] = useState('');
    const [servicePrice, setServicePrice] = useState('');
    const [serviceDuration, setServiceDuration] = useState('');
    const [serviceDescription, setServiceDescription] = useState('');
```

```
    const [message, setMessage] = useState('');


    const handleServiceUpload = async (e) => {
        e.preventDefault();
        if (!serviceName || !servicePrice || !serviceDuration || !serviceDescription) {
            setMessage(T('errorServiceFields'));
            return;
        }


        const newService = {
            serviceName,
            servicePrice: parseFloat(servicePrice),
            serviceDuration: parseInt(serviceDuration, 10),
            serviceDescription,
            vendorId: userId, // CRUCIAL: Store the user ID of the service vendor
            timestamp: Date.now()
        };


        try {
            await addDoc(collection(db, SERVICES_COLLECTION_PATH), newService);

            setMessage(`Successfully uploaded service: ${serviceName}`);
            setServiceName('');
            setServicePrice('');
            setServiceDuration('');
            setServiceDescription('');


        } catch (error) {
            console.error("Error adding service:", error);
            setMessage(`Error adding service: ${error.message}`);
        }
    };


    return (
        <div className="bg-white p-6 rounded-xl shadow-lg border-t-4 border-yellow-500">
            <h3 className="text-xl font-semibold mb-4 text-gray-700">{T('serviceUpload')}</h3>

            <p className="text-green-600 font-medium mb-4">{message}</p>


            <form onSubmit={handleServiceUpload} className="space-y-4">
                <input type="text" placeholder={T('serviceName')} value={serviceName} onChange={(e)
=> setServiceName(e.target.value)} className="w-full p-3 border border-gray-300 rounded-lg"
required />
                <textarea placeholder={T('serviceDescription')} value={serviceDescription}
onChange={(e) => setServiceDescription(e.target.value)} className="w-full p-3 border border-
gray-300 rounded-lg h-24" required />
```

```
            <div className="flex gap-4">
              <input type="number" placeholder={T('servicePrice')} value={servicePrice}
onChange={(e) => setServicePrice(e.target.value)} className="w-1/2 p-3 border border-gray-300
rounded-lg" step="0.01" required />
              <input type="number" placeholder={T('serviceDuration')} value={serviceDuration}
onChange={(e) => setServiceDuration(e.target.value)} className="w-1/2 p-3 border border-
gray-300 rounded-lg" required />
            </div>

            <button
              type="submit"
              className="w-full bg-yellow-500 text-white p-3 rounded-lg font-semibold hover:bg-
yellow-600 transition duration-200"
            >
              {T('uploadService')}
            </button>
          </form>
        </div>
    );
 };



 // Main Shopkeeper Dashboard component
 const ShopkeeperDashboard = () => {
    const [mode, setMode] = useState('shop'); // 'shop', 'ticket', 'service', or 'orders'


    return (
      <div className="p-6">
        <h2 className="text-3xl font-bold text-indigo-700 mb-6 flex items-center">
          <Briefcase className="w-8 h-8 mr-2" /> {T('shopManagement')}
        </h2>


        <div className="flex justify-center mb-6 space-x-4 flex-wrap gap-2">
          <button
            onClick={() => setMode('shop')}
            className={`py-2 px-4 rounded-full font-semibold transition duration-200 ${
              mode === 'shop' ? 'bg-indigo-500 text-white shadow-md' : 'bg-gray-200 text-
gray-700 hover:bg-gray-300'
            }`}
          >
            {T('manageProducts')}
          </button>
          <button
            onClick={() => setMode('ticket')}
            className={`py-2 px-4 rounded-full font-semibold transition duration-200 ${
              mode === 'ticket' ? 'bg-red-500 text-white shadow-md' : 'bg-gray-200 text-gray-700
hover:bg-gray-300'
            }`}
          >
            {T('manageTickets')}
```

```
            </button>
            <button
              onClick={() => setMode('service')}
              className={`py-2 px-4 rounded-full font-semibold transition duration-200 ${
                mode === 'service' ? 'bg-yellow-500 text-white shadow-md' : 'bg-gray-200 text-
gray-700 hover:bg-gray-300'
              }`}
            >
              {T('manageServices')}
            </button>
            <button
              onClick={() => setMode('orders')}
              className={`py-2 px-4 rounded-full font-semibold transition duration-200 ${
                mode === 'orders' ? 'bg-green-500 text-white shadow-md' : 'bg-gray-200 text-
gray-700 hover:bg-gray-300'
              }`}
            >
              {T('manageOrders')}
            </button>
          </div>


          {mode === 'shop' ? <ShopProductManagementForm />
            : mode === 'ticket' ? <TicketManagementForm />
            : mode === 'service' ? <AppointmentManagementForm />
            : <OrderManagement /> // Render Order Management
          }
        </div>
    );
};



// --- Shop Listing Logic (Replaces ShopsBrowser for filtered view) ---


// Takes a category string (e.g., 'Food Court' or 'NonFood') to filter
const ShopListing = ({ filter }) => {
    const [selectedSubCategory, setSelectedSubCategory] = useState('All');
    const [searchTerm, setSearchTerm] = useState(''); // NEW SEARCH STATE


    // Non-food categories for the filter bar (used only when filter is 'NonFood')
    // Now includes 'Snacks & Beverages'
    const allNonFoodCategories = ShopCategories.filter(cat => cat !== 'Food Court');

    // 1. Filter the shopsData based on the explicit 'filter' prop and search term
    const filteredShops = shopsData.filter(shop => {
        const shopCategory = shop.category || 'General Merchandise';
        const lowerCaseSearch = searchTerm.toLowerCase();


        // Filter 1: Main View Filter (based on filter prop)
```

```
        let passesMainFilter = false;

        if (filter === 'NonFood') {
            // Main Shops View: Exclude ONLY the 'Food Court' category
            passesMainFilter = shopCategory !== 'Food Court';
        } else {
            // Food Views: Show ONLY the 'Food Court' category
            passesMainFilter = shopCategory === filter;
        }

        if (!passesMainFilter) return false;


        // Filter 2: Sub-Category Filter (only applicable to NonFood view)
        if (filter === 'NonFood' && selectedSubCategory !== 'All') {
            if (shopCategory !== selectedSubCategory) return false;
        }


        // Filter 3: Item Search Filter (must have at least one available item matching the search
term)
        // Only apply this filter if a search term is present
        if (lowerCaseSearch !== '') {
            const itemsMatch = shop.items?.some(item =>
                item.available && item.name.toLowerCase().includes(lowerCaseSearch)
            );
            return itemsMatch;
        }


        // If no search term, pass the filter
        return true;
    });


    // 2. Group the filtered shops by category
    const categorizedShops = filteredShops.reduce((acc, shop) => {
        const category = shop.category || 'General';
        if (!acc[category]) {
            acc[category] = [];
        }
        acc[category].push(shop);
        return acc;
    }, {});


    // Determine title based on filter
    const titleKey = filter === 'Food Court' ? 'exploreCourt' :
                     'exploreShops'; // This now includes snacks


    return (
        <div className="p-6">
```

```
        <h2 className="text-3xl font-bold text-indigo-700 mb-6">{T(titleKey)}</h2>

        {/* --- Search Bar --- */}
        <div className="relative mb-6">
          <Search className="absolute left-3 top-1/2 transform -translate-y-1/2 w-5 h-5 text-
gray-400" />
          <input
            type="text"
            placeholder={T('searchPlaceholder')}
            value={searchTerm}
            onChange={(e) => setSearchTerm(e.target.value)}
            className="w-full p-3 pl-10 border border-gray-300 rounded-xl focus:ring-indigo-500
focus:border-indigo-500 transition duration-150 shadow-md"
          />
        </div>
        {/* --- End Search Bar --- */}



        {/* Category Filter Bar for Non-Food Shops (and now Snacks & Beverages) */}
        {filter === 'NonFood' && (
          <div className="flex flex-wrap gap-2 mb-8 justify-center">
            {/* 'All' Category Button */}
            <button
              key="All"
              onClick={() => { setSelectedSubCategory('All'); setSearchTerm(''); }} // Clear search
on category change
              className={`py-2 px-4 text-sm font-semibold rounded-full transition duration-200
                ${selectedSubCategory === 'All'
                  ? 'bg-indigo-600 text-white shadow-md'
                  : 'bg-gray-200 text-gray-700 hover:bg-gray-300'
                }`}
            >
              {T('allCategories')}
            </button>

            {/* All Non-Food Categories (including Snacks & Beverages) */}
            {allNonFoodCategories.map(cat => (
              <button
                key={cat}
                onClick={() => { setSelectedSubCategory(cat); setSearchTerm(''); }} // Clear
search on category change
                className={`py-2 px-4 text-sm font-semibold rounded-full transition
duration-200
                  ${selectedSubCategory === cat
                    ? 'bg-indigo-600 text-white shadow-md'
                    : 'bg-gray-200 text-gray-700 hover:bg-gray-300'
                  }`}
              >
                {getCategoryTranslation(cat, language)}
              </button>
            ))}
          </div>
        )}
```

```
          {Object.keys(categorizedShops).length === 0 ? (
            <div className="p-8 text-center bg-gray-100 rounded-xl">
              <AlertTriangle className="w-8 h-8 mx-auto mb-4 text-gray-500" />
              <p className="text-xl font-medium text-gray-700">{T('noShopsTitle')}</p>
              <p className="text-gray-500 mt-2">{T('noShopsHint')}</p>
            </div>
          ) : (
            // Always show the category heading
            Object.keys(categorizedShops).sort().map(category => (
              <div key={category} className="mb-8">
                <h3 className="text-2xl font-bold text-gray-800 border-b-2 border-indigo-400 pb-2
mb-4">{getCategoryTranslation(category, language)}</h3>
                <div className="space-y-6">
                  {categorizedShops[category].map(shop => (
                    <ShopCard key={shop.id} shop={shop} handleAddToCart={handleAddToCart}
searchTerm={searchTerm} /> // PASS SEARCH TERM to ItemCard Filter
                  ))}
                </div>
              </div>
            ))
          )}
        </div>
    );
 };


 const ShopCard = ({ shop, handleAddToCart, searchTerm }) => {
    // Only Food Court is considered 'Food' now for icon purposes
    const isFood = shop.category === 'Food Court';
    const Icon = isFood ? ChefHat : ShoppingBag;

    // Filter items based on availability AND the search term
    const lowerCaseSearch = searchTerm.toLowerCase();
    const availableItems = shop.items?.filter(item =>
      item.available !== false &&
        lowerCaseSearch === '' || item.name.toLowerCase().includes(lowerCaseSearch)
      )
    ) || [];


    // If no items match, don't display the shop card (this is defensive since filteredShops already
does this)
    if (availableItems.length === 0) return null;


    return (
      <div className="bg-white p-5 rounded-xl shadow-xl border border-gray-200
hover:shadow-2xl transition duration-300">
        <h4 className="text-xl font-extrabold text-indigo-600 mb-3 flex items-center">
          <Icon className={`w-5 h-5 mr-2 ${isFood ? 'text-green-500' : 'text-indigo-500'}`} />
```

```
{shop.shopName}
                <span className="ml-auto text-sm font-medium text-gray-500 bg-gray-100 px-3 py-1
rounded-full">{getCategoryTranslation(shop.category, language)}</span>
            </h4>

            {availableItems.length > 0 ? (
                <div className="grid grid-cols-2 sm:grid-cols-3 md:grid-cols-4 gap-4 mt-4">
                    {availableItems.map((item, index) => (
                        <ItemCard key={index} shop={shop} item={item}
handleAddToCart={handleAddToCart} />
                    ))}
                </div>
            ) : (
                // This message should only appear if the search filtered out all items, but the shop
passed the initial filter.
                <p className="text-gray-500 italic mt-3">{T('noItems')}</p>
            )}
        </div>
    );
};


const ItemCard = ({ shop, item, handleAddToCart }) => (
    <div className="flex flex-col bg-gray-50 rounded-lg overflow-hidden shadow-md">
        <img
            // FIX: Re-add prefix for display
            src={`${item.photoUrl}`}
            alt={item.name}
            className="w-full h-24 object-cover object-center"
            onError={(e) => {
                // Fallback to a placeholder if the URL is broken (or Base64 is too large/invalid)
                e.target.onerror = null;
                e.target.src = `https://placehold.co/100x100/3B82F6/white?text=
${item.name.substring(0, 10)}`;
            }}
        />
        <div className="p-3">
            <p className="text-sm font-semibold text-gray-800 truncate">{item.name}</p>
            <p className="text-lg font-bold text-green-600 mt-1">₹{item.price ?
item.price.toFixed(2) : 'N/A'}</p>
            <button
                onClick={() => handleAddToCart(shop, item)}
                className="mt-2 w-full bg-indigo-500 text-white text-sm py-1.5 rounded-lg font-
semibold hover:bg-indigo-600 transition duration-200"
            >
                {T('addToCart')}
            </button>
        </div>
    </div>
);


// --- Events Browser Logic (User View - Tickets) ---
const EventsBrowser = () => {
```

```
    const sortedTickets = [...ticketsData].sort((a, b) => new Date(a.eventDate) - new
Date(b.eventDate));


    return (
      <div className="p-6">
        <h2 className="text-3xl font-bold text-red-700 mb-6">{T('exploreEvents')}</h2>

        {sortedTickets.length === 0 ? (
          <div className="p-8 text-center bg-gray-100 rounded-xl">
            <Ticket className="w-8 h-8 mx-auto mb-4 text-red-500" />
            <p className="text-xl font-medium text-gray-700">{T('noEventsTitle')}</p>
            <p className="text-gray-500 mt-2">{T('noEventsHint')}</p>
          </div>
        ) : (
          <div className="grid grid-cols-1 md:grid-cols-2 gap-6">
            {sortedTickets.map(ticket => (
              <EventTicketCard key={ticket.id} ticket={ticket}
handleAddToCart={handleAddToCart} />
            ))}
          </div>
        )}
      </div>
    );
  }


  const EventTicketCard = ({ ticket, handleAddToCart }) => {
    // Simple date formatting
    const dateOptions = { year: 'numeric', month: 'short', day: 'numeric' };
    const formattedDate = new Date(ticket.eventDate).toLocaleDateString(language,
dateOptions);

    // Define ticket as an 'item' object for the cart system
    const ticketItem = {
      name: `${ticket.eventName} Ticket`,
      price: ticket.ticketPrice,
    };


    const handleBuyTicket = () => {
      // Add the ticket item to the cart
      // We pass placeholder shop/item structure (TICKET_SALES) and the organizer's ID for
fulfillment
      handleAddToCart(
        { id: 'TICKET_SALES', shopName: 'Event Tickets' },
        ticketItem,
        ticket.organizerId // Pass organizer ID as fulfillment ID
      );
    };


    return (
```

```
        <div className="bg-white p-5 rounded-xl shadow-xl border-t-4 border-red-500 flex flex-
col">
            <img
              src={ticket.photoUrl}
              alt={ticket.eventName}
              className="w-full h-40 object-cover object-center rounded-lg mb-4"
              onError={(e) => {
                e.target.onerror = null;
                e.target.src = `https://placehold.co/400x160/DC2626/white?text=
${ticket.eventName.substring(0, 15)}`;
              }}
            />
            <h4 className="text-2xl font-extrabold text-red-600 mb-3">{ticket.eventName}</h4>

            <div className="space-y-2 text-gray-700 text-sm mb-4">
              <div className="flex items-center">
                <Calendar className="w-4 h-4 mr-2 text-indigo-500" />
                <span className="font-semibold">{formattedDate}</span>
              </div>
              <div className="flex items-center">
                <Clock className="w-4 h-4 mr-2 text-indigo-500" />
                <span>{ticket.eventTime}</span>
              </div>
              <div className="flex items-center">
                <MapPin className="w-4 h-4 mr-2 text-indigo-500" />
                <span>{ticket.eventLocation}</span>
              </div>
            </div>


            <div className="mt-auto pt-3 border-t border-gray-100 flex items-center justify-
between">
              <p className="text-xl font-bold text-green-600">₹{ticket.ticketPrice ?
ticket.ticketPrice.toFixed(2) : 'N/A'}</p>
              <button
                onClick={handleBuyTicket}
                className="bg-red-500 text-white py-2 px-4 rounded-lg font-semibold hover:bg-
red-600 transition duration-200"
              >
                {T('buyTicket')}
              </button>
            </div>
          </div>
      );
  }

  // NEW: Appointment Browser Logic (User View - Services)
  const AppointmentBrowser = () => {
      return (
          <div className="p-6">
              <h2 className="text-3xl font-bold text-yellow-700 mb-6 flex items-center">
                  <Calendar className="w-8 h-8 mr-2" />{T('exploreAppointments')}
              </h2>
```

```
          {servicesData.length === 0 ? (
            <div className="p-8 text-center bg-gray-100 rounded-xl">
              <ListPlus className="w-8 h-8 mx-auto mb-4 text-yellow-500" />
              <p className="text-xl font-medium text-gray-700">{T('noShopsTitle')}</p>
              <p className="text-gray-500 mt-2">Vendors can upload bookable services via their portal.</p>
            </div>
          ) : (
            <div className="grid grid-cols-1 md:grid-cols-2 gap-6">
              {servicesData.map(service => (
                <ServiceCard key={service.id} service={service} handleAddToCart={handleAddToCart} />
              ))}
            </div>
          )}
        </div>
    );
 };


 const ServiceCard = ({ service, handleAddToCart }) => {

    const handleBookService = () => {
      // Define service as an 'item' object for the cart system
      const serviceItem = {
        name: `${service.serviceName} (${service.serviceDuration} ${T('mins')})`,
        price: service.servicePrice,
      };

      // Add the service item to the cart
      // We pass placeholder shop/item structure (BOOKABLE_SERVICES) and the vendor's ID for fulfillment
      handleAddToCart(
        { id: 'BOOKABLE_SERVICES', shopName: 'Bookable Services' },
        serviceItem,
        service.vendorId // Pass vendor ID as fulfillment ID
      );
    };

    return (
      <div className="bg-white p-5 rounded-xl shadow-xl border-t-4 border-yellow-500 flex flex-col">
        <h4 className="text-2xl font-extrabold text-yellow-600 mb-3 flex items-center">
          <Clock className="w-5 h-5 mr-2" /> {service.serviceName}
        </h4>

        <p className="text-gray-600 mb-4 flex-grow text-sm">{service.serviceDescription}</p>


        <div className="space-y-2 text-gray-700 text-sm mb-4 border-t pt-3">
          <div className="flex items-center justify-between">
            <span className="font-semibold">{T('servicePrice')}:</span>
            <p className="text-xl font-bold text-green-600">₹{service.servicePrice ? service.servicePrice.toFixed(2) : 'N/A'}</p>
```

```
            </div>
            <div className="flex items-center justify-between">
              <span className="font-semibold">{T('serviceDuration')}:</span>
              <span className="text-md font-medium text-gray-800">{service.serviceDuration}
{T('mins')}</span>
            </div>
          </div>


          <button
            onClick={handleBookService}
            className="bg-yellow-500 text-white py-2 px-4 rounded-lg font-semibold hover:bg-
yellow-600 transition duration-200"
          >
            {T('bookNow')}
          </button>
        </div>
    );
 };

 // NEW: Cart View Component
 const CartView = () => {
    const total = cart.reduce((sum, item) => sum + item.price * item.quantity, 0);


    return (
      <div className="p-6">
        <h2 className="text-3xl font-bold text-indigo-700 mb-6 flex items-center">
          <ShoppingBag className="w-8 h-8 mr-2" /> {T('viewCart')}
        </h2>


        {cart.length === 0 ? (
          <div className="p-8 text-center bg-yellow-100 rounded-xl">
            <AlertTriangle className="w-8 h-8 mx-auto mb-4 text-yellow-500" />
            <p className="text-xl font-medium text-gray-700">{T('cartEmpty')}</p>
            <button
              onClick={() => setCurrentView(Views.CUSTOMER_PORTAL)}
              className="mt-4 bg-indigo-500 text-white py-2 px-4 rounded-lg font-semibold
hover:bg-indigo-600 transition duration-200"
            >
              {T('startShopping')}
            </button>
          </div>
        ) : (
          <div className="space-y-4">
            {cart.map((item) => (
              <div key={item.uniqueId} className="flex items-center justify-between bg-white
p-4 rounded-xl shadow-md border-l-4 border-indigo-400">
                <div className="flex-grow">
                  <p className="text-lg font-semibold text-gray-800">{item.name}</p>
                  <p className="text-sm text-gray-500">From: {item.shopName}</p>
                </div>
```

```
                    <div className="flex items-center space-x-4">
                      {/* Quantity Controls */}
                      <div className="flex items-center border border-gray-300 rounded-lg
overflow-hidden">
                        <button
                          onClick={() => handleRemoveOneFromCart(item.uniqueId)} // Decrement
or Remove
                          className="px-3 py-1 text-red-500 hover:bg-red-100 transition
duration-150 font-bold text-lg"
                          aria-label="Decrease quantity"
                        >
                          -
                        </button>
                        <span className="px-3 font-semibold text-gray-800 border-l border-r
border-gray-300">
                          {item.quantity}
                        </span>
                        <button
                          // Re-use handleAddToCart logic, passing the necessary shop/item
details
                          onClick={() => handleAddToCart(
                            { id: item.shopId, shopName: item.shopName },
                            { name: item.name, price: item.price }
                          )}
                          className="px-3 py-1 text-green-500 hover:bg-green-100 transition
duration-150 font-bold text-lg"
                          aria-label="Increase quantity"
                        >
                          +
                        </button>
                      </div>
                      <span className="text-xl font-bold text-green-600">₹{(item.price *
item.quantity).toFixed(2)}</span>
                    </div>
                  </div>
                ))}

                <div className="flex justify-between items-center p-4 bg-indigo-100 rounded-xl
shadow-inner border-t-2 border-indigo-500 mt-6">
                  <p className="text-sm font-semibold text-gray-700 flex items-center">
                    <DollarSign className="w-4 h-4 mr-2 text-green-600"/> {T('paymentMethod')}
                  </p>
                  <span className="text-base font-bold text-green-700">{T('cod')}</span>
                </div>

                <div className="flex justify-between items-center p-4 bg-indigo-100 rounded-xl
shadow-inner border-t-2 border-indigo-500">
                  <p className="text-2xl font-bold text-indigo-700">{T('cartTotal')}</p>
                  <p className="text-3xl font-extrabold text-green-700">₹{total.toFixed(2)}</p>
                </div>



                <button
```

```
                    onClick={handlePlaceOrder}
                    disabled={!customerProfile}
                    className="w-full bg-green-500 text-white p-3 rounded-lg font-semibold text-lg
hover:bg-green-600 transition duration-200 disabled:bg-green-300"
                >
                    {T('placeOrder')} ({!customerProfile ? 'Login Required' : `₹${total.toFixed(2)}`})
                </button>
                {!customerProfile && (
                    <p className="text-red-500 text-center text-sm mt-2">{T('errorMissingProfile')}</
p>
                )}
            </div>
        )}
    </div>
  );
};

// NEW: Order Confirmation View Component
const OrderConfirmationView = () => {
  if (!lastPlacedOrder) {
    return (
      <div className="p-6 text-center text-red-500">
        <p>No order details found. Please place an order first.</p>
        <button
          onClick={() => setCurrentView(Views.CUSTOMER_PORTAL)}
          className="mt-4 bg-indigo-500 text-white py-2 px-4 rounded-lg font-semibold
hover:bg-indigo-600"
        >
          {T('backToPortal')}
        </button>
      </div>
    );
  }

  const order = lastPlacedOrder;

  return (
    <div className="max-w-xl mx-auto p-6 bg-white rounded-xl shadow-2xl border-t-8 border-
green-500">
      <div className="text-center mb-6">
        <CheckCircle className="w-12 h-12 mx-auto mb-3 text-green-600" />
        <h2 className="text-3xl font-bold text-green-700">{T('orderPlacedSuccessfully')}</
h2>
        <p className="text-lg text-gray-600 mt-2">{T('orderSummaryTitle')}</p>
      </div>


      {/* Summary Box */}
      <div className="space-y-4 p-4 border border-gray-200 rounded-lg bg-gray-50">
        <div className="flex justify-between items-center text-sm">
          <span className="font-semibold text-gray-700">{T('orderNumber')}</span>
          <span className="font-mono text-gray-800">{order.id.substring(0, 10)}</span>
        </div>
        <div className="flex justify-between items-center text-sm border-t pt-2">
```

```
              <span className="font-semibold text-gray-700 flex items-center"><DollarSign
className="w-4 h-4 mr-2 text-green-600"/> {T('paymentMethod')}</span>
              <span className="font-bold text-green-700">{order.paymentMethod}</span>
            </div>
            <div className="flex justify-between items-center text-sm">
              <span className="font-semibold text-gray-700 flex items-center"><Clock
className="w-4 h-4 mr-2 text-indigo-500"/> {T('estimatedDelivery')}</span>
              <span className="font-bold text-indigo-700">{T('deliveryTime')}</span>
            </div>
          </div>

          {/* Delivery Info */}
          <h3 className="text-xl font-bold text-indigo-700 mt-6 mb-3 flex items-center border-b
pb-1">
            <MapPin className="w-5 h-5 mr-2" /> {T('deliveryInfo')}
          </h3>
          <div className="text-gray-700 space-y-1 text-base">
            <p className="font-semibold">{order.customerName}</p>
            <p>{order.customerAddress}</p>
            <p className="text-sm font-mono text-gray-600">{order.customerPhone}</p>
          </div>


          {/* Item Breakdown */}
          <h3 className="text-xl font-bold text-indigo-700 mt-6 mb-3 flex items-center border-b
pb-1">
            <ListPlus className="w-5 h-5 mr-2" /> {T('itemBreakdown')}
          </h3>
          <div className="space-y-3">
            {order.itemsByShop.map((shopGroup, index) => (
              <div key={index} className="border-l-4 border-indigo-200 pl-3 pt-1">
                <p className="font-bold text-base text-gray-800 mb-1">{shopGroup.shopName}
</p>
                {shopGroup.items.map((item, itemIndex) => (
                  <div key={itemIndex} className="flex justify-between text-sm text-gray-600">
                    <span>{item.quantity} x {item.name}</span>
                    <span>₹{(item.price * item.quantity).toFixed(2)}</span>
                  </div>
                ))}
              </div>
            ))}
          </div>


          {/* Final Total */}
          <div className="flex justify-between items-center mt-6 pt-4 border-t-2 border-
green-500">
            <p className="text-2xl font-bold text-indigo-700">{T('cartTotal')}</p>
            <p className="text-3xl font-extrabold text-green-700">₹{order.totalAmount.toFixed(2)}
</p>
          </div>

          <button
            onClick={() => setCurrentView(Views.CUSTOMER_PORTAL)}
```

```
            className="w-full mt-6 bg-indigo-500 text-white p-3 rounded-lg font-semibold text-lg hover:bg-indigo-600 transition duration-200"
          >
            {T('backToPortal')}
          </button>
        </div>
    );
};


const CustomerPortal = () => (
    <div className="p-6">
        <h2 className="text-2xl font-semibold text-gray-700 mb-6">{T('selectService')}</h2>
        <div className="grid grid-cols-1 sm:grid-cols-2 gap-6">
            <CategoryCard
                icon={ShoppingBag}
                title={T('shops')}
                onClick={() => setCurrentView(Views.SHOPS)}
                color="bg-indigo-600"
            />
            <CategoryCard
                icon={Ticket}
                title={T('eventTickets')}
                onClick={() => setCurrentView(Views.TICKETS)}
                color="bg-red-500"
            />
            <CategoryCard
                icon={Utensils}
                title={T('food')} // Now reads "Food Court"
                onClick={() => setCurrentView(Views.COURT_LISTING)} // Navigates directly to Food Court listing
                color="bg-green-600"
            />
            <CategoryCard
                icon={Calendar}
                title={T('appointment')}
                onClick={() => setCurrentView(Views.APPOINTMENT)}
                color="bg-yellow-500"
            />
        </div>
    </div>
);



// --- Main View Renderer ---


const renderView = () => {
    switch (currentView) {
        case Views.DASHBOARD:
            return <AccessScreen />;
        case Views.CUSTOMER_PORTAL:
```

```
            return <CustomerPortal />;
        case Views.SHOPS:
            // Filter: Show all non-food shops (including Snacks & Beverages)
            return <ShopListing filter="NonFood" />;
        case Views.COURT_LISTING:
            // Filter: Show only Food Court shops
            return <ShopListing filter="Food Court" />;
        case Views.TICKETS:
            return <EventsBrowser />;
        case Views.SHOPKEEPER_DASHBOARD:
            return <ShopkeeperDashboard />;
        case Views.APPOINTMENT:
            return <AppointmentBrowser />;
        case Views.CART:
            return <CartView />;
        case Views.ORDER_CONFIRMATION: // NEW VIEW
            return <OrderConfirmationView />;
        default:
            return <div className="p-6 text-center text-red-500">View not found.</div>;
    }
  };


  if (loading) {
    return <LoadingState />;
  }


  return (
    <div className="min-h-screen bg-gray-100 font-sans">
      <Header title={T('title')} />
      <main className="max-w-4xl mx-auto py-8">
        {renderView()}
      </main>
    </div>
  );
};


export default App;
```